Somewhat more usefully, you can create a mapped type with a generic type rather than with a specific type:

```
type Partial<T> = { [P in keyof T]?: T[P] }
type OptionalTicketData = Partial<TicketData>

# or you can skip the declaration
const x: Partial<TicketData>
```

I think that you are unlikely to be creating these in your day-to-day life, but TypeScript defines a few of them,[76] and I can see a few being useful now and again, like `Readonly<T>` (which makes all the properties of the type read only) or `NonNullable<T>` (which constructs a new type excluding `null` or `undefined`) or `ReturnType<T>` (which resolves to the return type of a functional type and which we saw back in Chapter 12, *Using Redux to Manage State*).